

ARQUITECTURA PROPUESTA PARA LA EMPRESA BP

Requerimiento: Sistema de Banca por Internet

Objetivo:

Generar un sistema de banca por internet que permite hacer lo siguiente:

- Consulta de Movimientos Históricos
- Transferencias y Pagos entre Cuentas Propias e Interbancaria

Actores: Usuario.

Canales identificados: Web y Móvil

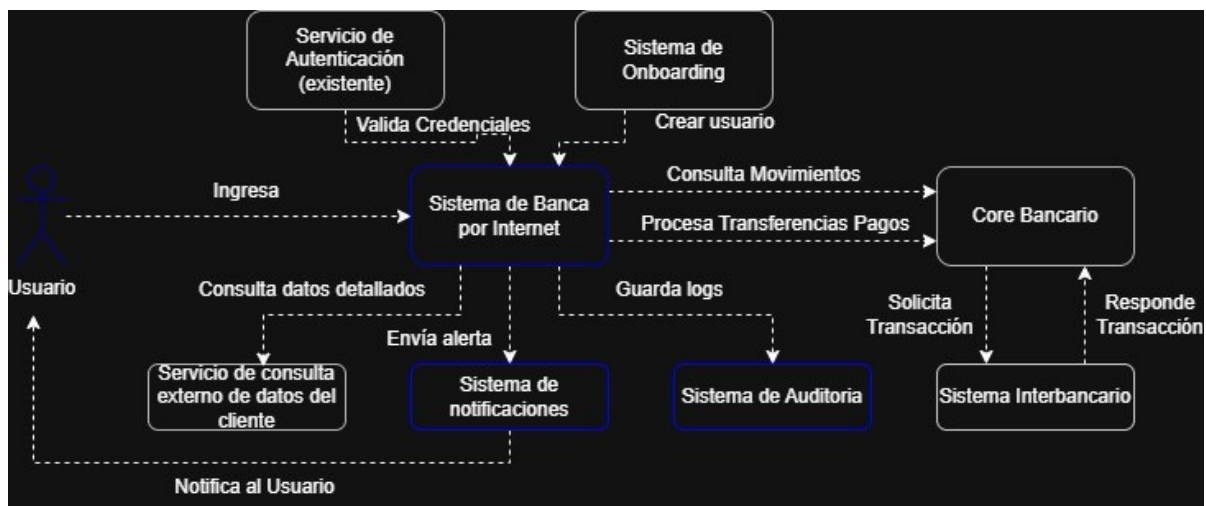
Flujos identificados:

- Autenticación y Autorización de Usuarios
- Onboarding Nuevos Usuarios
- Consulta de información de clientes
- Consulta de movimientos
- Transferencias y Pagos
- Envío de notificaciones al usuario
- Logs de Auditoria

Requerimientos no funcionales:

- Protección de datos personales
- Alta Disponibilidad
- Tolerancia a Fallos
- Recuperación ante desastres
- Seguridad Operativa
- Excelencia operativa y autohealing

Diagrama de Contexto:



Leyenda:

Azul: sistemas nuevos o afectados/usuario

Gris: Sistemas existentes

Justificación:

- Se considera los siguientes sistemas como externos al sistema de banca por internet debido a que algunos son existentes como el caso del servicio de autenticación, el sistema de onboarding, el core

bancario. Otros aún no existen o van a ser consumidos de proveedores externos como el caso del sistema de notificaciones que puede ser propio o a través de un proveedor, el sistema interbancario que es un sistema externo que conecta para realizar las transacciones con otros bancos y el sistema de consulta externo de datos detallados del cliente que de acuerdo al requerimiento puede realizarse si se requiere desde una aplicación externa.

El sistema de auditoria se menciona la existencia de una base de datos pero no de un sistema como tal para que sea específico para registrar estos logs por lo que se ha considerado como para modificación.

El diagrama ilustra la arquitectura de un sistema de banca por internet, organizado en capas y componentes interconectados:

- Usuario (Persona):** El punto de entrada principal que interactúa con el sistema.
- Frontend:** Incluye el **Web SPA** (Contenedor: Javascript/ Angular) y el **App Móvil** (Contenedor: Android/ iOS).
- API Gateway:** Actúa como intermediario, validando credenciales usando el **Servicio de Autenticación** (OAuth2.0) y redirigiendo a nuevos usuarios al **Sistema de Onboarding** (Sistema Onboarding).
- BFF (Backend for Frontend):** Incluye **BFF Web** (Contenedor: .NET 8 Web API) y **BFF Móvil** (Contenedor: .NET 8 Web API), que consumen microservicios usando [e.g. JSON/HTTPS].
- Microservicios:** Se dividen en:
 - Microservicio Canales** (Contenedor: .NET 8 WebAPI/gRPC)
 - Microservicio Productos** (Contenedor: .NET 8 WebAPI/gRPC)
 - Microservicio Transacciones** (Contenedor: .NET 8 WebAPI/gRPC)
- Base de Datos y Cache:** Incluye **Cache Redis** (Contenedor: Redis) y **Base de datos** (Contenedor: SqServer).
- Servicio de Consulta Externa de datos del cliente:** Proporciona datos detallados usando [JSON/HTTPS] y escribe datos usando [EF Core / TDS Puerto 1433].
- Microservicio de Integración:** (Contenedor: .NET 8 WebAPI) que consume integración usando [JSON/HTTPS].
- Core Bancario:** (Core Bancario) que transacciona usando [JSON/HTTPS] con el **Sistema Interbancario** (Sistema Interbancario).
- Actualizar Worker Service:** (C#, .NET 8) que lee cola usando [AMQP/TCP] y envía información actualizar usando [json/TCP].
- Broker de mensajes core bancario:** (Contenedor: Apache Kafka / RabbitMQ) que envía información actualizar usando [json/TCP].
- Auditoria Worker Service:** (C#, .NET 8) que lee cola usando [AMQP/TCP] y guarda log usando [ADO.NET / TCP] o [Driver Nativo / TCP].
- Auditoria:** (Sistema de Auditoria) que guarda log usando [ADO.NET / TCP] o [Driver Nativo / TCP].
- Notificaciones Worker Service:** (C#/Background Service) que graba alertas usando [ADO.NET / TCP] o [Driver Nativo / TCP] y procesa notificaciones usando [HTTPS/Azure SDK] o [HTTPS/JSON/XML].
- Base de Datos Notificaciones:** (SQLServer/NoSQL).
- Servidor Corporativo Email OnPremise:** (C#/ .NET 8).
- Monitoreo y Logging:** Incluye **Azure Application Insights** (Sistema Externo/SaaS) que recibe logs de errores, métricas y excepciones usando [HTTPS/gRPC], y **Azure Communication Services** (Sistema Externo/Service Cloud) que envía notificaciones.

Azul: Sistemas afectados
Celeste: Usuario
Gris: Sistemas externos

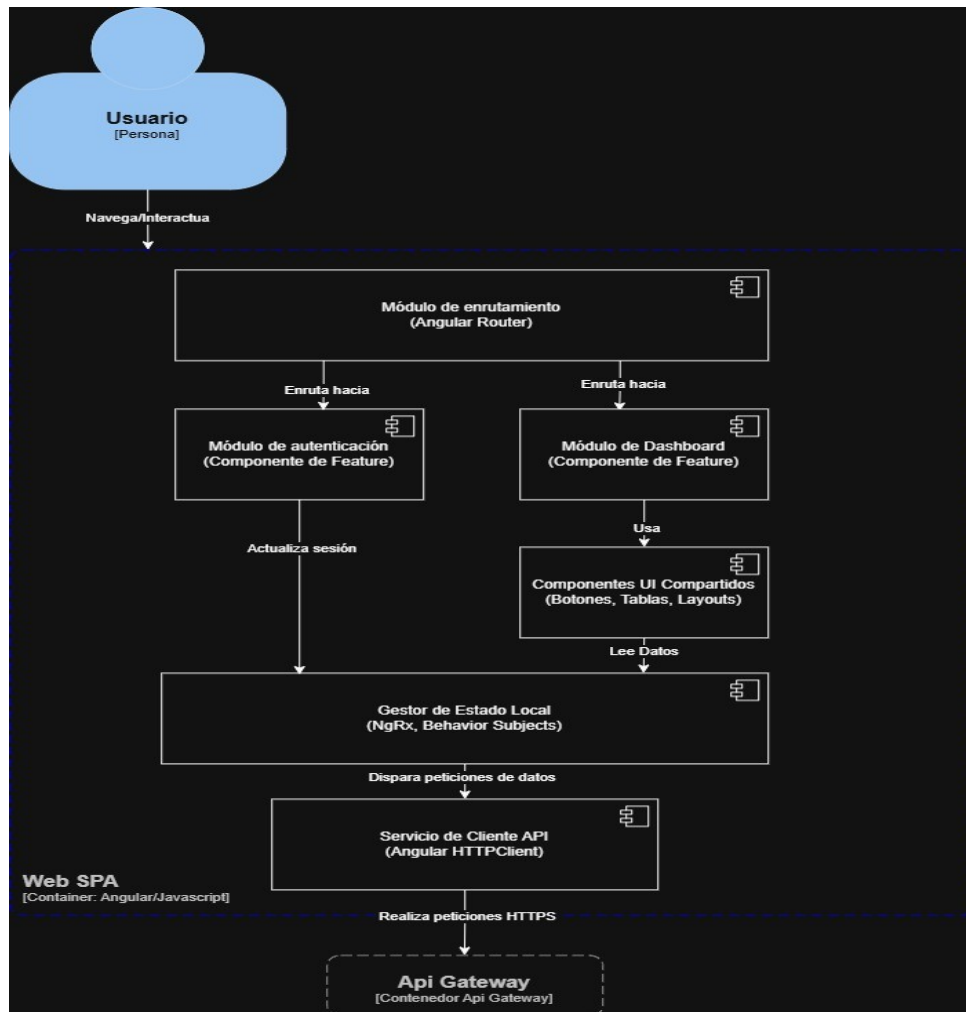
Justificación:

- Se ha optado por utilizar una arquitectura híbrida con los siguientes detalles:

- Arquitectura de microservicios limpia y desacoplada para que cambios futuros no afecten lo que ya existe.
- Un patrón de puerta de enlace (Api Gateway): Para que nos permita tener centralizado y controlado el acceso a los diferentes componentes, sus políticas de acceso y autenticación y de igual manera su balanceo en caso de tener picos de tráfico.
- Un patrón Backend For FrontEnd para separar las peticiones de cada canal, permitiendo optimizar las peticiones, el formateo de datos y la seguridad según las necesidades particulares de cada plataforma antes de comunicarse con los microservicios internos.
- Una arquitectura Event-Driven o dirigida por eventos para comunicar los microservicios con componentes como el *Auditoria Worker Service* y el *Notificaciones Worker Service* y tener un desacoplamiento asíncrono. Los sistemas reaccionan a eventos en lugar de depender únicamente de llamadas síncronas.
- Un patrón CQRS con la presencia de una *Cache Redis* para la validación de llaves y consulta de datos detallados introduce una estrategia de optimización de lecturas frente a las escrituras directas en las bases de datos transaccionales.
- Se ha generado dos sistemas satélite de notificaciones y de auditoría ambos consumiendo desde un broker de mensajes a través de un worker service los datos respectivos para guardar en base de datos y en el caso de notificaciones enviarle también al cliente. Se hizo de esta manera para tener desacoplado estos temas del flujo transaccional ya que en caso de fallas de cualquiera de estos sistemas o saturación no se afecten todos al mismo tiempo y se pueda seguir transaccionando.
- Se usa una comunicación o protocolo HTTPS entre los aplicativos web y móvil, los bff's y los microservicios para conexiones seguras y cifrar la información sensible evitando el robo de información

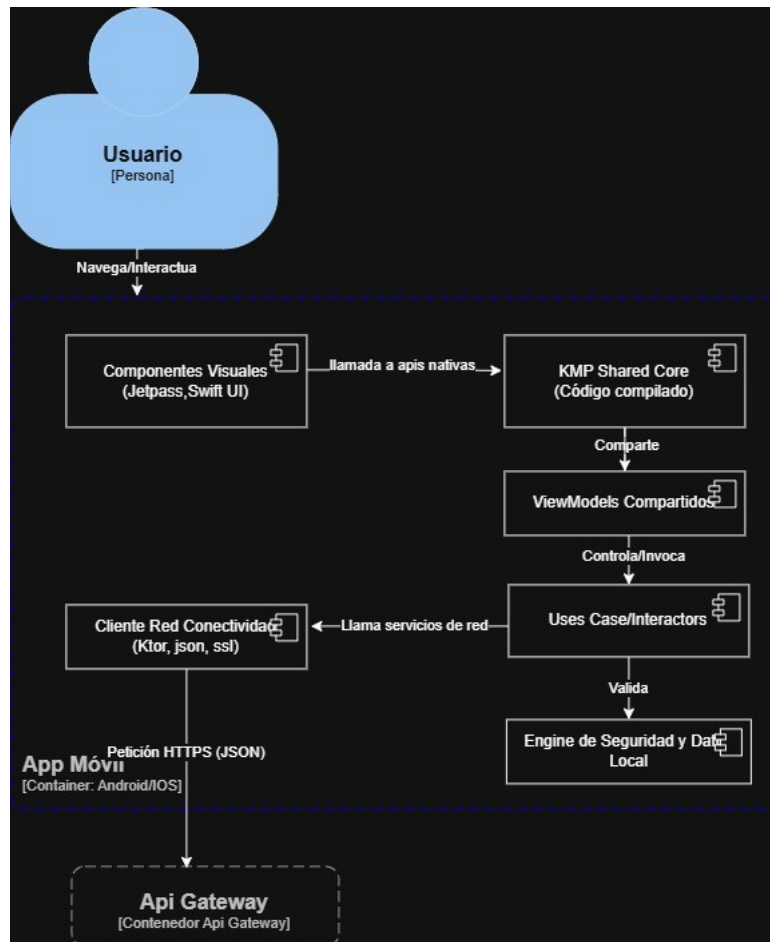
Diagrama de Componentes Banca por Internet (Sistema Bancario):

Contenedor Web SPA:



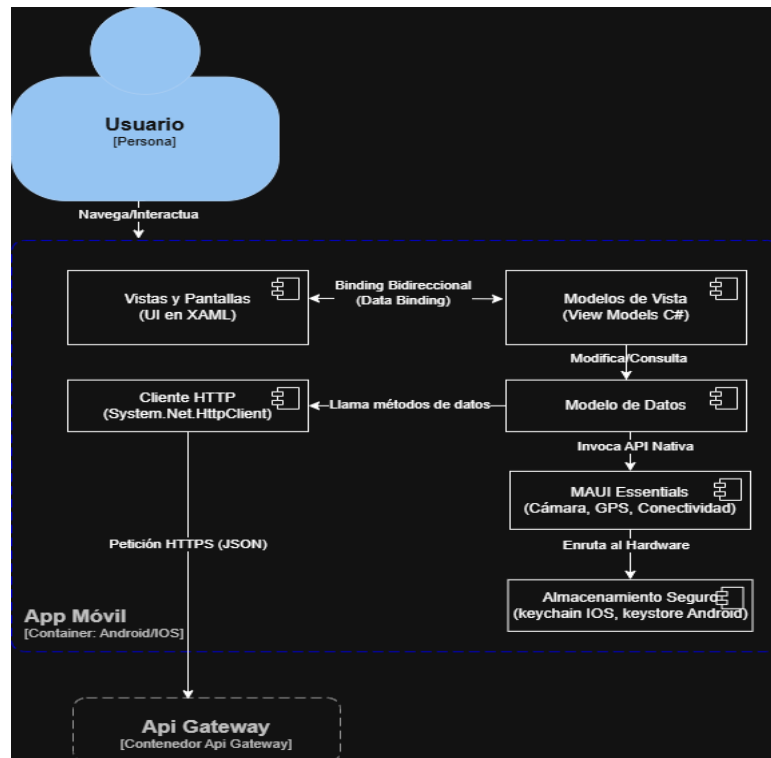
Para la aplicación web se ha elegido como herramienta angular por su versatilidad y facilidad para generar nuevas funcionalidades debido a su arquitectura modular. De igual manera proporciona estándares robustos de seguridad y capacidad para manejar sistemas complejos a gran escala. Al ser una aplicación SPA que es una aplicación de una sola página todas las demás funcionalidades se cargan de manera dinámica lo que hace que tenga mayor velocidad y fluidez y sea excelente para transaccionar.

Contenedor App Móvil: Opción con Kotlin



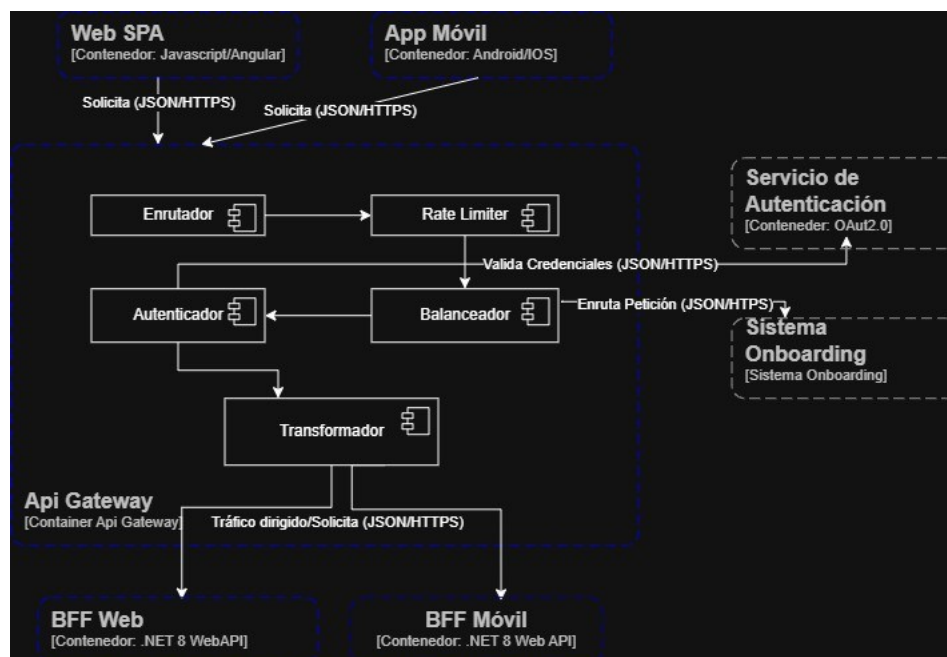
Se ha considerado Kotlin para el desarrollo de la app móvil ya que es multiplataforma tiene un gran respaldo de Google por el tema de android y adicional puede ser desarrollada en java que al momento tiene un gran soporte. Proporciona alta estabilidad de los datos y seguridad fuerte. Permite integración con sistemas de autenticación biométrica como huella dactilar y reconocimiento facial. Tiene encriptación de datos tanto en el dispositivo como en tránsito y no necesita que se genere código por separado para android y IOS ya que la compilación genera los ejecutables para cada uno de estos sistemas operativos móviles, por lo que es una gran opción para el desarrollo de esta app.

Opción con .Net MAUI



Al igual que Kotlin, .NET MAUI permite desarrollar aplicaciones multiplataforma usando un mismo código que al compilar genera los ejecutables tanto para Android como iOS. Proporciona seguridad y rendimiento ya que al compilarse en código nativo ofrece velocidad y rendimiento necesario para el procesamiento de transacciones. De igual manera se integra de forma sencilla con sistemas de biometría como huella dactilar y reconocimiento facial. Cuenta con el respaldo de Microsoft por lo que la hace una gran opción para el desarrollo de esta app móvil.

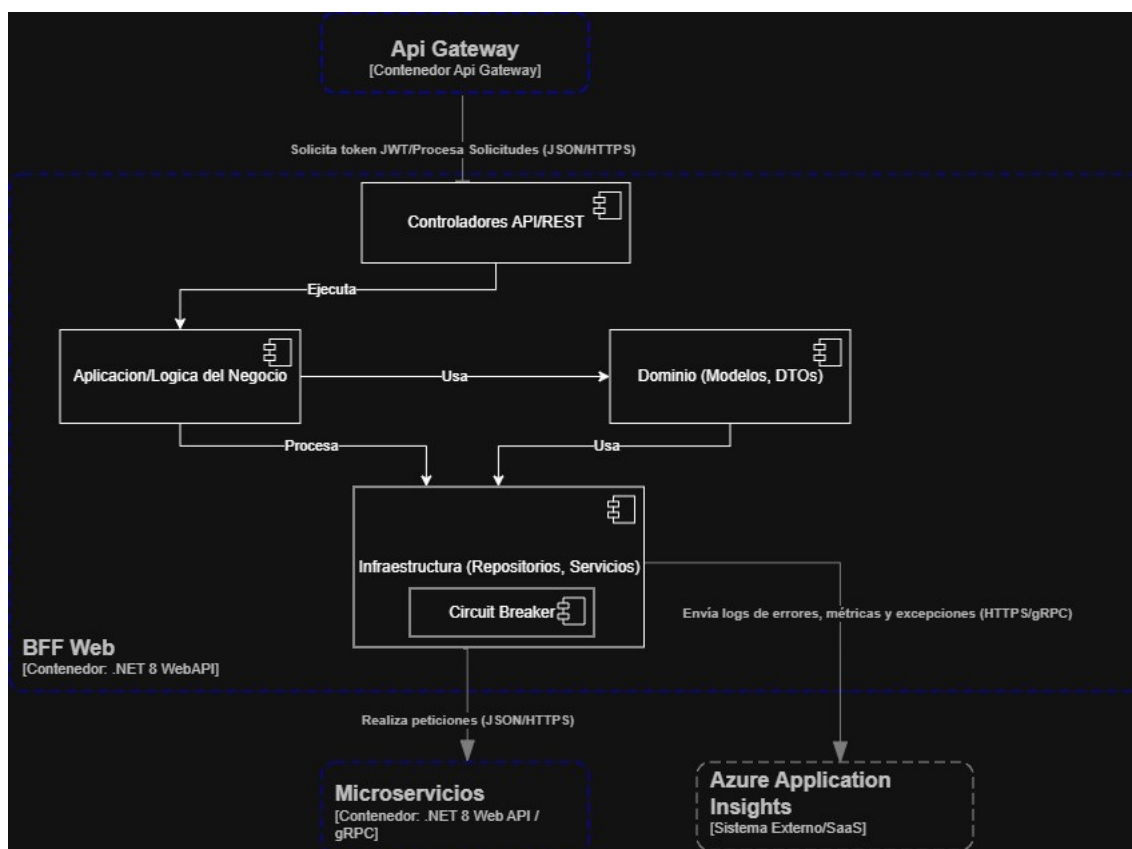
Contenedor API Gateway:



Se ha utilizado el api gateway para centralizar el tema de autenticación y políticas de acceso. Al validar las credenciales contra el servicio de autenticación existente Auth2.0, si es correcta proporcionará un token jwt que el api gateway propagará hacia los bff's para poder continuar transaccionando en los diferentes endpoints. Este api gateway podría ser uno que nos proporciona azure como el azure api management que nos proporciona el escalado automático lo que permite balancear las peticiones. De igual forma proporciona seguridades como el rate limit en caso de ataques externos para que no se degrade el consumo de las aplicaciones.

Para el tema del onboarding se ha hecho que se redireccione desde el api gateway para evitar el acoplamiento con otras aplicaciones.

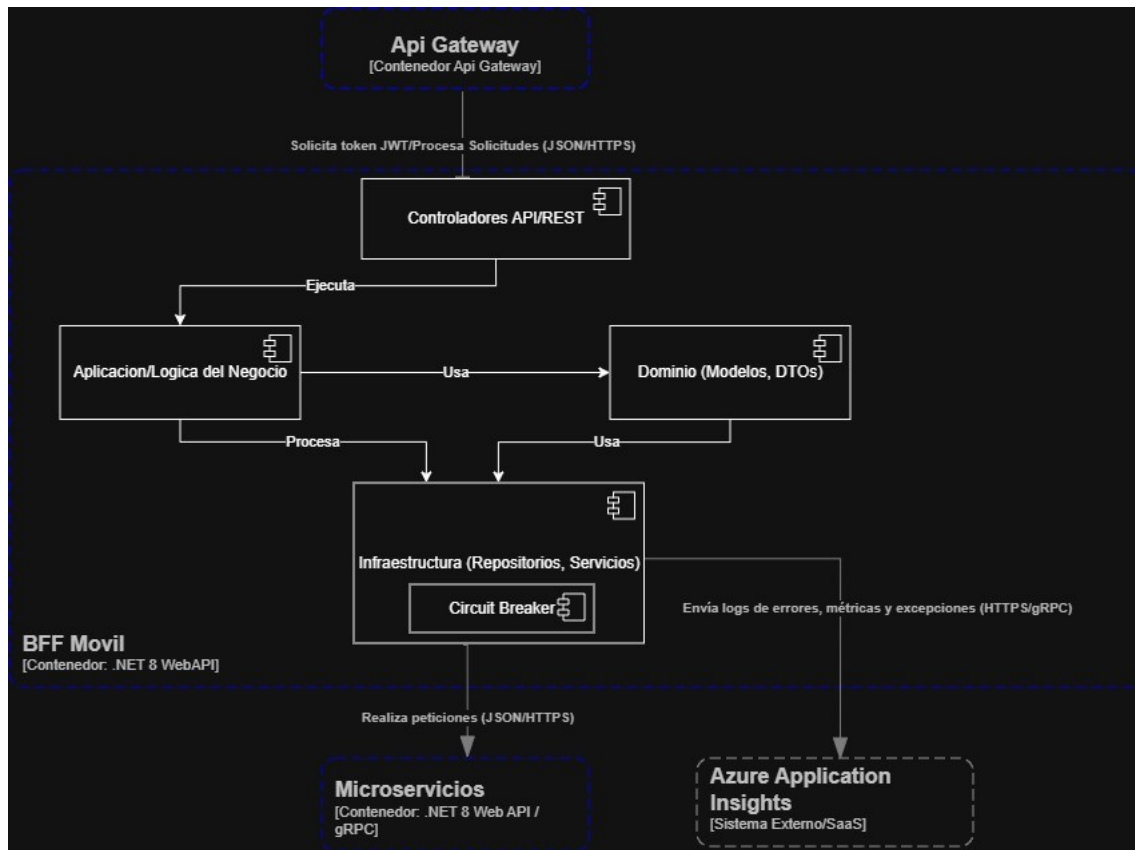
Contenedor Backend For FrontEnd Web (BFF Web)



Este contenedor es parte del patrón Backend For FrontEnd para separar las peticiones por canal y se lo ha realizado para atender específicamente las peticiones del canal Web con eso se puede formatear los datos que requiere este canal y evitar datos innecesarios que no se requieren y que podrían degradar el uso de la aplicación web. Con esto también se deja desacoplado el uso de los microservicios por lo que si en el futuro se crean nuevos pueden ser consumidos sin problema por el bff sin afectar lo existente. Adicional se ha mapeado que este bff envíe sus logs de errores, métricas al Application Insights que es un servicio en la nube para poder monitorear los sistemas en tiempo real.

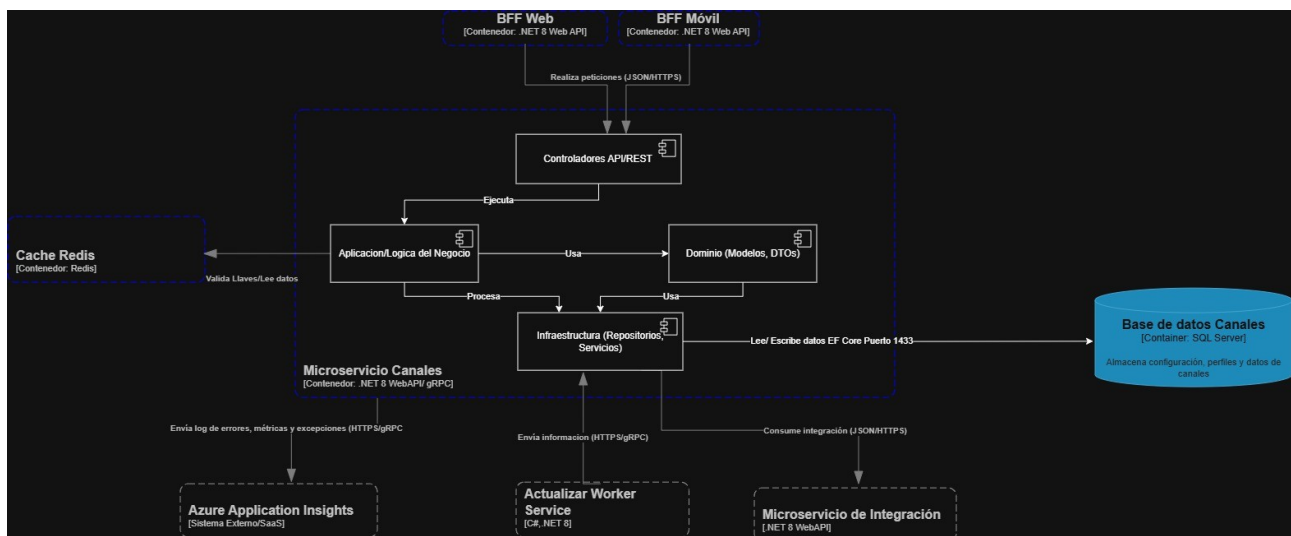
De igual manera se ha implementado un patrón de desarrollo de arquitectura limpia para separar la presentación, la lógica del negocio, el dominio y la infraestructura con el propósito que si se cambia de proveedor o la librería o microservicio cambia no se afecte el funcionamiento de lo existente. En la capa de infraestructura se ha colocado un *circuit breaker* para en caso de fallos de algún microservicio o componente externo no se detenga el flujo hacia otros microservicios.

Contenedor Backend For FrontEnd Móvil (BFF Movil)



Al igual que el anterior es parte del patrón Backend For FrontEnd y se lo ha realizado para atender las peticiones del canal Móvil con esto se formatea los datos específicos para este canal y se evita devolver datos que sobrecarguen la app móvil. Al igual que el anterior envía sus logs de errores, métricas hacia el Application Insights para el tema de monitoreo y supervisión. También tiene el patrón de desarrollo de arquitectura limpia e incorpora el circuit braker en caso de fallo de algún componente externo no se degrade la transaccionalidad.

Contenedor Microservicio Canales:



Este microservicio se creo para procesar las solicitudes y cargar configuraciones y perfiles para los canales existentes al momento y consume su base de datos cumpliendo con el principio de Base de

Datos por Servicio para desacoplarse de otros componentes y hacer más eficiente el tema de rendimiento.

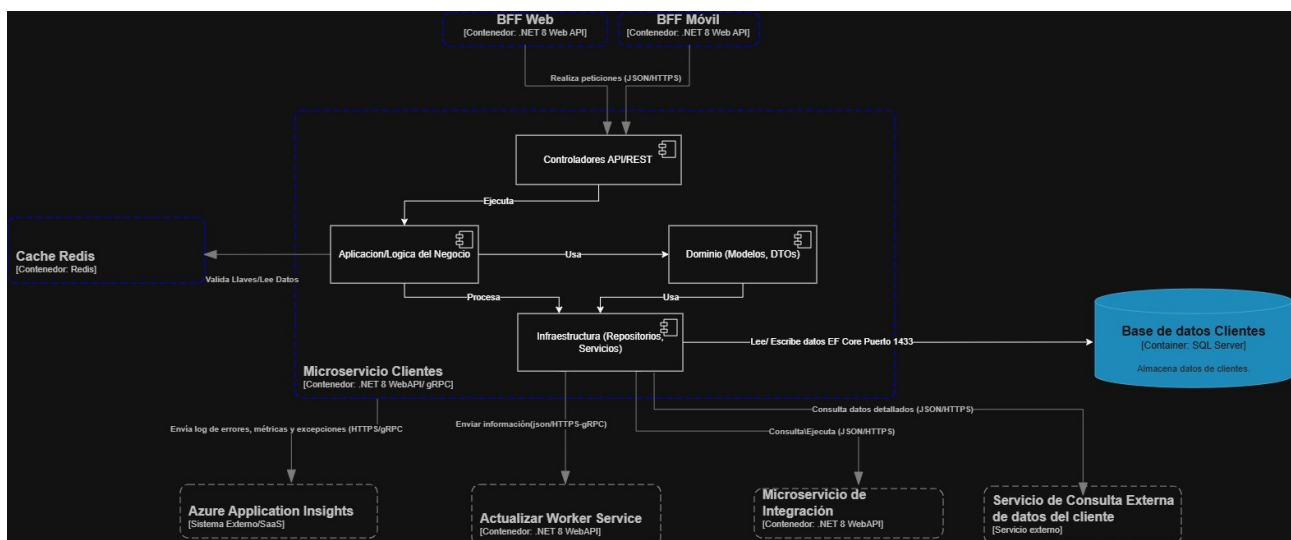
Este al igual que los otros microservicios se conecta a un componente CacheRedis para la lectura de llaves y optimización de lecturas directas desde la Base de Datos.

Al igual que los Bff's usa un patrón de desarrollo de arquitectura limpia para separar las capas de presentación, aplicación o lógica de negocio, dominio e infraestructura para no afectar la lógica del negocio en caso de algún cambio de proveedor o librería.

Este componente también deja los logs de error, métricas en el Application Insights para poder ser spuervisado y monitoreado.

Se usa el Microservicio de integración como intermediario para realizar las peticiones asíncronas al core bancario y evitar posibles caídas de todo el sistema a causa del core. De igual manera se usa el ActualizarWorkerService para recibir de manera asíncrona los datos que se cambien en el core bancario y actualizar a nivel de base de datos de canales.

Contenedor Microservicio Clientes:



Este microservicio se creó para procesar las peticiones referentes a los clientes y consume su base de datos propia de Clientes cumpliendo con el principio de Base de Datos por Servicio para desacoplarse de otros componentes y hacer más eficiente el tema de rendimiento.

También se conecta a un componente CacheRedis lectura de llaves y optimización de lecturas directas desde la Base de Datos.

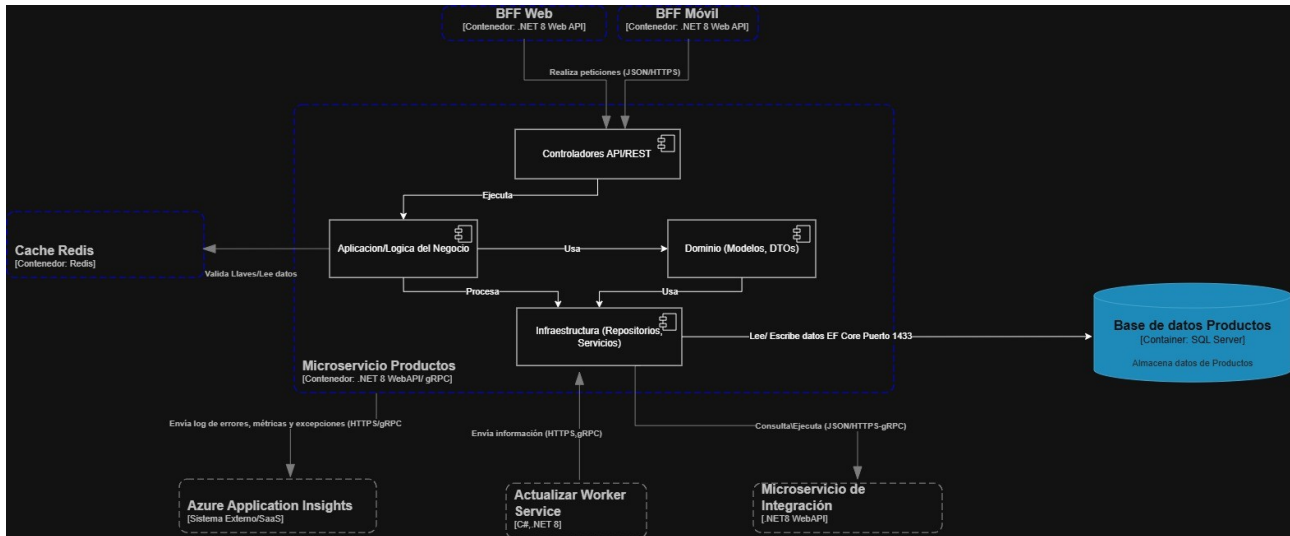
Se usa el Microservicio de integración como intermediario para realizar las peticiones asíncronas al core bancario y evitar posibles caídas de todo el sistema a causa del core. De igual manera se usa el ActualizarWorkerService para recibir de manera asíncrona los datos que se cambien en el core bancario y actualizar a nivel de base de datos de clientes. Se conecta al Servicio de Consulta Externa para obtener los datos detallados de los clientes. Estos datos los agrupa y los devuelve al bff cuando se solicite consultar la información del cliente.

Se ha pensado que es la mejor solución acceder directamente desde este microservicio al servicio externo para consulta de datos detallados del cliente ya que se evita una llamada adicional desde el lado del cliente: una para consultar los datos básicos y luego otra para los datos detallados, por lo que en la llamada para consultar los datos del cliente el microservicio accederá al core bancario y al servicio unificando las respuestas en un solo modelo

Usa un patrón de desarrollo de arquitectura limpia para separar las capas de presentación, aplicación o lógica de negocio, dominio e infraestructura para no afectar la lógica del negocio en caso de algún cambio de proveedor o librería.

Hace uso del Application Insights para enviar los logs de errores y métricas para el tema de monitoreo y supervisión.

Contenedor Microservicio Productos:



Este microservicio se creó para procesar las peticiones de Productos y consume su base de datos propia cumpliendo con el principio de Base de Datos por Servicio para desacoplarse de otros componentes y hacer más eficiente el tema de rendimiento.

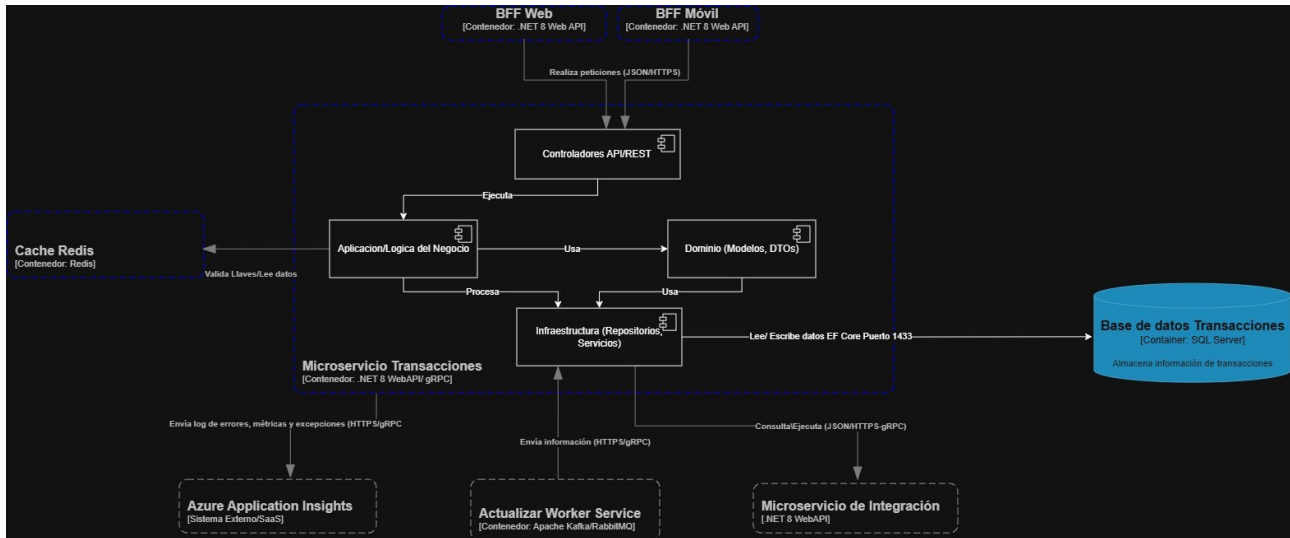
Igual que el anterior se conecta a un componente CacheRedis para la lectura de llaves y optimización de lecturas directas desde la Base de Datos.

Se usa el Microservicio de integración como intermediario para realizar las peticiones asíncronas al core bancario y evitar posibles caídas de todo el sistema a causa del core. De igual manera se usa el ActualizarWorkerService para recibir de manera asíncrona los datos que se cambien en el core bancario y actualizar a nivel de base de datos de productos.

Usa un patrón de desarrollo de arquitectura limpia para separar las capas de presentación, aplicación o lógica de negocio, dominio e infraestructura para no afectar la lógica del negocio en caso de algún cambio de proveedor o librería.

Usa Application Insights para enviar los logs de errores y métricas para el tema de monitoreo y supervisión.

Contenedor Microservicio Transacciones:



Este microservicio se creó para procesar las peticiones de movimientos y transacciones de transferencias o pagos y consume su base de datos propia cumpliendo con el principio de Base de Datos por Servicio para desacoplarse de otros componentes y hacer más eficiente el tema de rendimiento.

Se conecta a un componente CacheRedis para la validación de llaves de idempotencia y optimización de lecturas directas desde la Base de Datos.

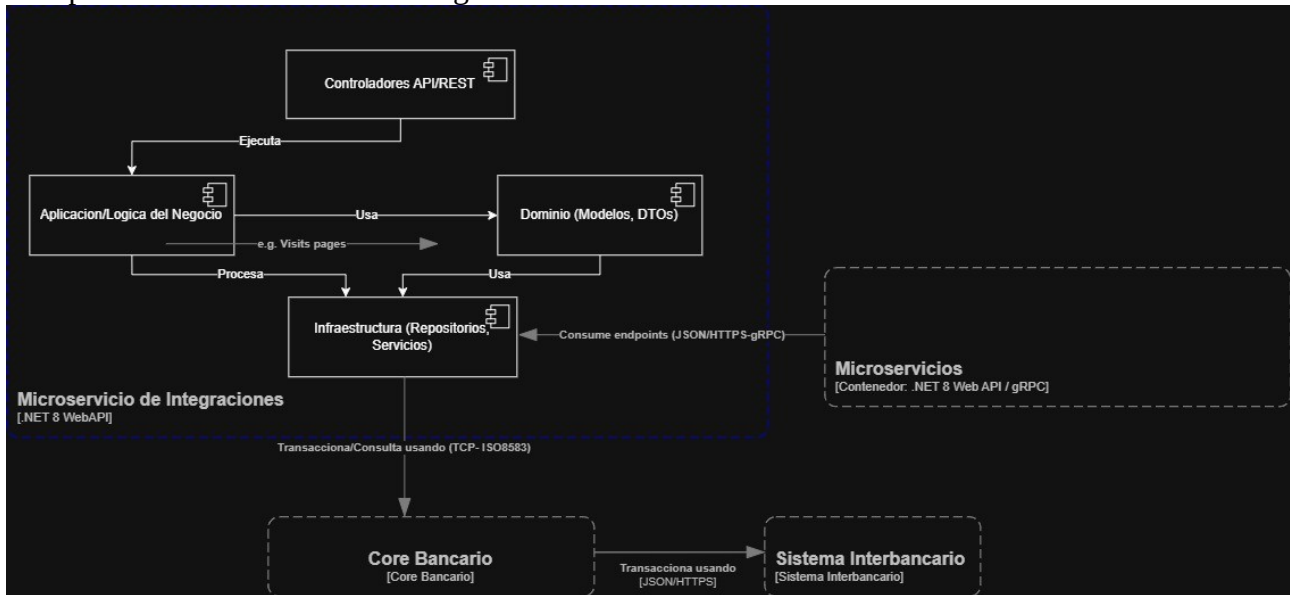
Se usa el Microservicio de integración como intermediario para realizar las peticiones asíncronas al core bancario y evitar posibles caídas de todo el sistema a causa del core. De igual manera se usa el ActualizarWorkerService para recibir de manera asíncrona los datos que se cambian en el core bancario y actualizar a nivel de base de datos de canales.

El core bancario cuando se realiza una transacción interbancaria se conecta al sistema interbancario para procesar dichas transacciones, por lo que no se ha conectado al microservicio de integración directo con ese servicio externo.

Usa un patrón de desarrollo de arquitectura limpia para separar las capas de presentación, aplicación o lógica de negocio, dominio e infraestructura para no afectar la lógica del negocio en caso de algún cambio de proveedor o librería.

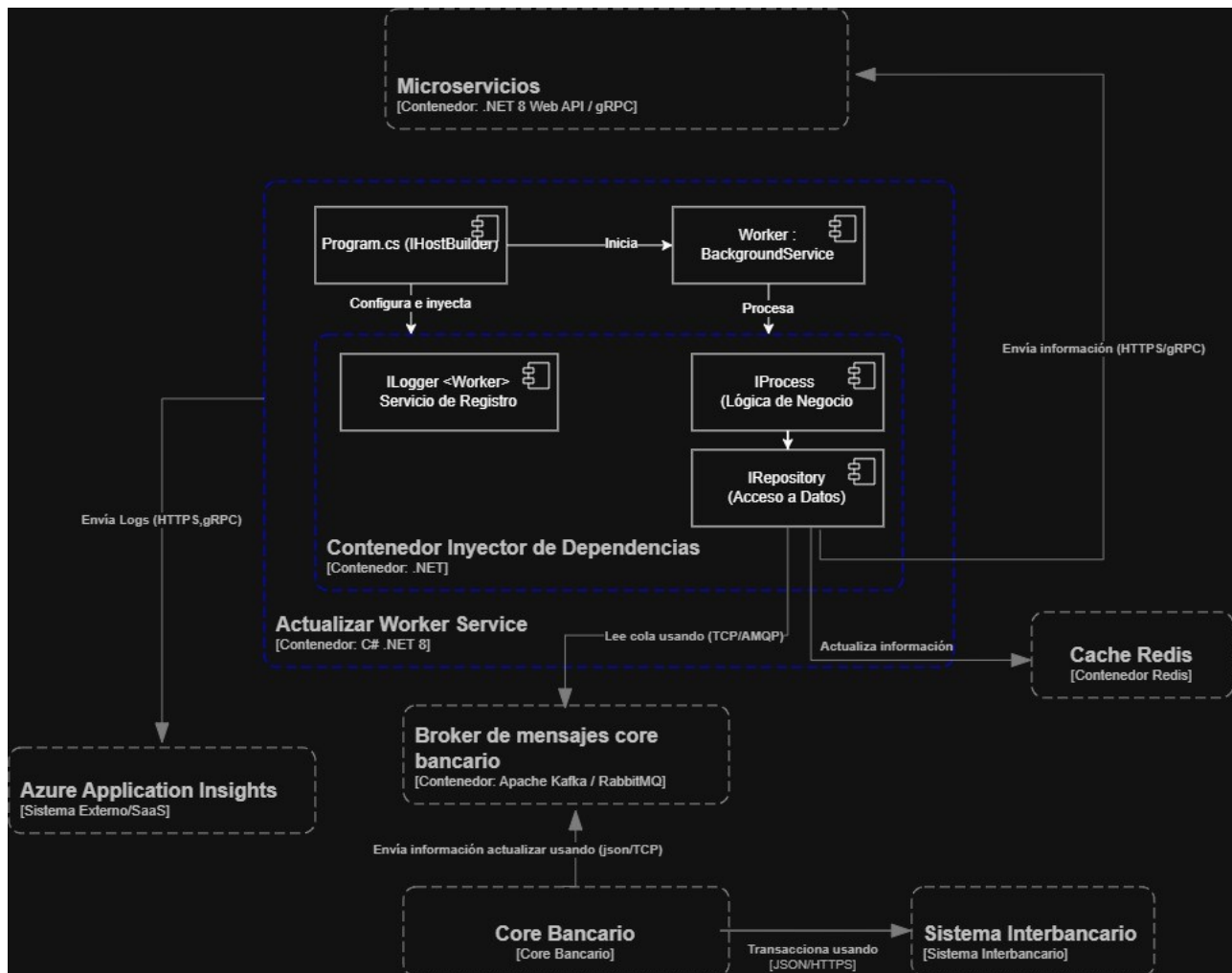
Usa Application Insights para enviar los logs de errores y métricas para el tema de monitoreo y supervisión.

Componente Microservicio de Integración:



Este microservicio como su nombre lo indica fue creado para servir de puente de conexión entre el core bancario que generalmente es un sistema legado y los microservicios, entonces lo que se hace es consumir desde los microservicio los endpoints de acuerdo a la consulta o transacción que se realice y el microservicio de integración transforma esta petición en una trama o lenguaje que maneja el core bancario para procesar la solicitud y devuelve este resultado para que el microservicio de integración lo devuelva al microservicio que lo solicitó.

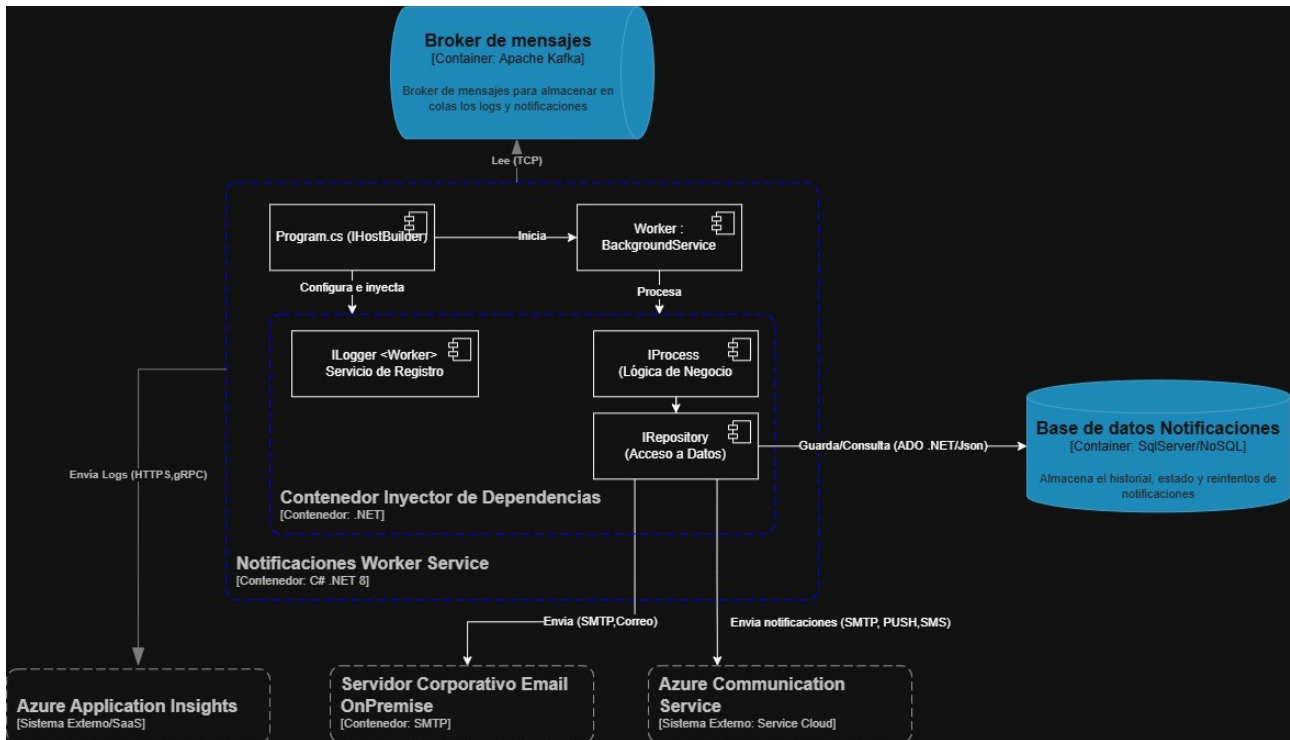
Componente Actualizar Worker Service:



Este worker service nos permite enviar de manera asíncrona la información que se actualiza en el core a los microservicios y estos a su vez poderla actualizar en sus bases de datos respectivas, de igual manera envía a actualizar en el redis las claves con los nuevos valores en el core bancario para que las lecturas de los microservicios se hagan con los últimos valores vigentes y de manera rápida. Para hacerlo el core bancario debe depositar la información en el broker de mensajes core bancario y de allí la tomará el Actualizar WorkerService de la cola respectiva. De igual manera guarda logs de errores y métricas en Azure Application Insights.

Diagrama de Componentes Sistema de Notificaciones.

Componente Notificacion Worker Service:



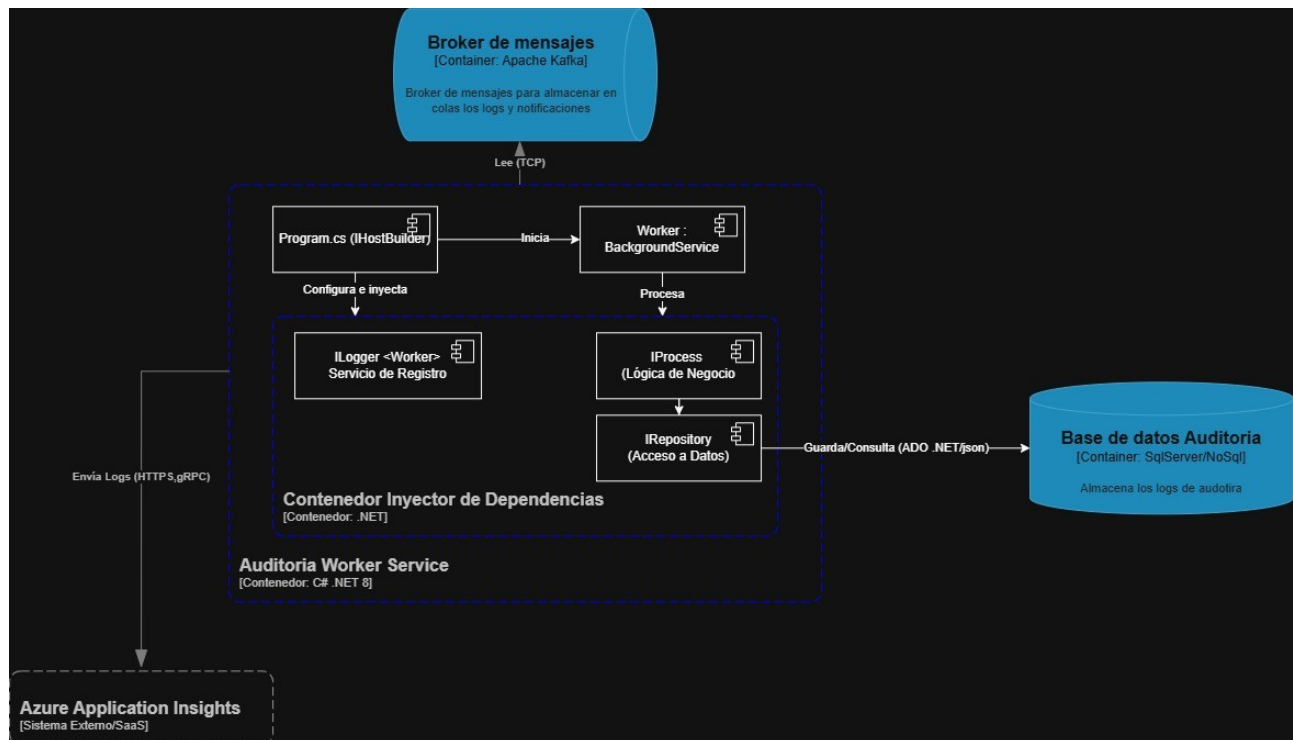
Se ha creado el sistema de notificaciones que contiene un Notificaciones Worker Service que es un servicio creado con C# y .NET 8 el mismo que lee las notificaciones de la cola del broker de mensajes específico para notificaciones y luego las envía al proveedor seleccionado en la nube o bien a un servicio local para el procesamiento de las mismas. También se ha creado una base de datos para las notificaciones ya que normativamente se requiere tener un respaldo de las mismas por si existen reclamos de los clientes.

Para el envío de notificaciones por medio de un servicio en la nube se ha propuesto el uso de Azure Communication Service por cuanto tiene la flexibilidad de enviar notificaciones de distintos tipos Email, SMS, PUSH y el costo no es tan elevado. Estos para que todos los componentes en la nube estén también bajo el mismo proveedor.

La otra opción que se ha planteado es el uso de un servidor de correo on premise inicialmente solo para envío de Email, para el tema del envío de las otras notificaciones como push o sms se podría usar librerías o consumir algún proveedor externo que no sea de la nube para esta clase de envíos, pero eso tendría un costo adicional.

Diagrama de Componentes Sistema de Auditoria

Componente Autorizacion Worker Service:



Se ha creado un sistema independiente de auditoria para almacenar los logs de auditoria al igual que una base de datos propia ya que no deben mezclarse con las base de datos transaccionales y debe evitarse que sean manipulados.

Dentro de este sistema esta el componente Auditoria Worker Service que es un worker service creado en C# y .NET 8 este trabaja por detrás sin una interfaz gráfica buscando si existen logs nuevos en la cola del broker de mensajes especifico para autorizaciones y lo almacena en la base de datos. De igual manera deposita los logs de errores y métricas en el ApplicationInsights para poderlo monitorear y supervisar.

DECISIONES:

- Usar para el tema de la app móvil: MAUI y Kotlin ya que sus frameworks permiten compartir la lógica de negocio y son multiplataforma orientadas a aplicaciones de alto rendimiento y seguridad como se especifica en los diagramas de componentes.
- Para el tema de notificaciones para la opción en la nube se usará Azure Communication Service por un tema de compatibilidad con los diferentes medios para enviar notificaciones como Email, Web, SMS, Push y otra razón es para que todos los componentes tengan el mismo proveedor en la nube. Para la opción on premise o propia se plantea usar provisionalmente un servidor de correo corporativo ya que las opciones de push y sms dependen de proveedores externos.
- Para la seguridad se usará el servicio existente propuesto para la autenticación a través del api gateway y una vez autenticado el api gateway propagará el JWT que reciba del servicio de autenticación como firma autorizada para transaccionar a través de los bff's y microservicios.

- Para el api gateway se usara uno proporcionado en la nube que proporciona la seguridad, escalabilidad, balanceo, control de ataques como es APIM Azure Api Management. Para no tener que usar un balanceador independiente si nuestro api gateway fuera on premise.
- Para el monitoreo y supervisión de los componentes se usará el servicio Azure Application Insights en la nube para tener centralizado todos los componentes e integrar los logs facilmente por medio de la librería en el desarrollo. Al tener presupuesto para ello se está optando por este servicio.
- Para el tema de las transacciones como pagos y transferencias se plantea el uso de claves de idempotencia para evitar transacciones repetidas esto se generará desde el cliente y se almacenará en el cache redis.
- Por temas de seguridad se debe encriptar los datos sensibles como números de cuenta, números de tarjeta, identificaciones, tokens, que puedan comprometer los datos de los clientes.
- Los otp's, claves, tokens no deben almacenarse a nivel de base de datos en claro para notificaciones ya que son datos sensibles.
- Por el tema de protección de datos personales no se debe mostrar datos del cliente como identificaciones, números de tarjeta, números de cuenta en reportes o pantallas que sean visibles para otras personas que no sea el cliente o funcionario del banco autorizado para revisar ese módulo.
- Los datos biométricos como imágenes de huellas u otros no se guardarán a nivel de base de datos por temas de protección de datos solamente se guardará un hash.
- Para el ingreso al aplicativo web se realizará por medio de un usuario y clave y para la app móvil se podrá realizar con identificación biométrica integrando los sistemas propios del sistema operativo del dispositivo con la app móvil.
- Para el tema de los logs de auditoria se hará una persistencia en tablas separadas outbox en cada una de las bases de microservicios para guardar los logs para que en caso de degradación o pérdida del CDC o del broker de mensajes no se pierdan estos logs de auditoria. En el caso del CDC o background service este atenderá los eventos de cambio de datos en las tablas outbox donde se guardan los logs de auditoria y enviará esos datos al broker de mensaje para que sean persistidos en la base de datos propia de auditoria.
- Se recomienda considerar migrar el formato del mensaje en el broker de JSON a Apache Avro. Al integrarse con un *Schema Registry*, Avro reduce drásticamente el tamaño del payload en la red, acelera la velocidad de Kafka y garantiza de manera estricta que un cambio en la estructura del log de un microservicio no rompa el *Auditoria Worker Service* en el destino.
- Para el tema de auto healing se recomienda empaquetar en kubernetes los bff's y microservicios para que estén protegidos para auto recuperación.